

Compte rendu – Catalog Service API

Yannis

2 juin 2026

Table des matières

1 Présentation du projet

Le projet **Catalog Service** est un micro-service développé avec **Django** et **Django REST Framework**. Il permet de gérer un catalogue de produits dans le cadre du projet FlexShop.

L'API expose des endpoints REST permettant de créer, consulter, modifier, supprimer, filtrer, rechercher et trier des produits. Une documentation OpenAPI est également fournie via Swagger UI.

2 Lien vers le dépôt Git

Le dépôt Git du projet est disponible à l'adresse suivante :

[A_COMPLETER_AVEC_LE_LIEN_DU_DEPOT_GIT](#)

À compléter avant rendu : remplacer `A_COMPLETER_AVEC_LE_LIEN_DU_DEPOT_GIT` par le lien réel du dépôt Git.

3 Technologies utilisées

- Python 3.12+
- Django 6.0.5
- Django REST Framework 3.17.1
- django-filter
- drf-spectacular pour la documentation OpenAPI
- SQLite pour la base de données locale de développement

4 Fonctionnalités réalisées

4.1 CRUD complet

L'API permet d'effectuer toutes les opérations CRUD sur les produits :

Méthode et end-point	Description
GET /api/v1/products/	Liste tous les produits avec pagination.
POST /api/v1/products/	Crée un nouveau produit.
GET /api/v1/products/{id}/	Récupère le détail d'un produit.
PATCH /api/v1/products/{id}/	Modifie partiellement un produit.
PUT /api/v1/products/{id}/	Remplace complètement un produit.
DELETE /api/v1/products/{id}/	Supprime un produit.

4.2 Pagination, filtrage, recherche et tri

La pagination est configurée globalement dans Django REST Framework avec une taille de page de 10 éléments.

L'API permet également :

- le filtrage par catégorie avec `?category=electronics` ;
- la recherche par nom ou description avec `?search=souris` ;
- le tri par prix, date de création ou stock avec `?ordering=price_cents`.

4.3 Endpoint custom avec @action

Un endpoint personnalisé a été ajouté avec le décorateur `@action` de Django REST Framework :

```
GET /api/v1/products/low-stock/?threshold=5
```

Cet endpoint retourne les produits dont le stock est inférieur ou égal au seuil indiqué. Il permet par exemple d'identifier rapidement les produits à réapprovisionner.

4.4 Validation métier customisée

Le serializer applique plusieurs règles métier :

- le prix doit être strictement positif ;
- le prix ne peut pas dépasser 99 999 999,99 euros ;
- le stock ne peut pas dépasser 100 000 unités ;
- le paramètre `threshold` de l'endpoint `low-stock` doit être un entier.

4.5 Prix stocké en centimes

Le prix est stocké en base de données sous forme d'entier avec le champ `price_cents`. Cette approche évite les erreurs d'arrondi liées aux nombres flottants et correspond à une pratique courante dans les applications financières.

L'API reste simple à utiliser : l'utilisateur envoie un prix au format décimal, par exemple 29.99, et le serializer le convertit automatiquement en 2999 centimes.

5 Conformité REST

Les URI respectent les bonnes pratiques REST :

- les ressources sont nommées au pluriel : `/products/` ;
- les URI ne contiennent pas de verbe ;
- les méthodes HTTP indiquent l'action à effectuer ;
- les codes de statut HTTP sont cohérents : 200, 201, 204, 400, 404.

6 Documentation OpenAPI et Swagger UI

La documentation OpenAPI est générée avec `drf-spectacular`. Elle est disponible via les endpoints suivants :

- Swagger UI : <http://localhost:8000/api/docs/>
- Schéma OpenAPI : <http://localhost:8000/api/schema/>

6.1 Capture d'écran de Swagger UI fonctionnel

La capture d'écran de Swagger UI doit être placée dans le dossier `rendu/images/` avec le nom :

```
swagger-ui.png
```

Une fois l'image ajoutée, elle apparaîtra ci-dessous dans le PDF.

```
Capture Swagger UI à ajouter : rendu/images/swagger-ui.png
```

FIGURE 1 – Swagger UI fonctionnel pour l'API Catalog Service.

7 Collection Postman

Une collection Postman exportée au format JSON est fournie avec le rendu :

```
rendu/catalog-service-postman-collection.json
```

Cette collection contient les requêtes principales suivantes :

- liste des produits ;
- création d'un produit ;
- récupération du détail d'un produit ;
- modification partielle ;
- suppression ;
- filtrage par catégorie ;
- recherche ;
- tri par prix en centimes ;
- endpoint custom `low-stock` ;
- récupération du schéma OpenAPI.

8 Tests unitaires

Le projet contient 7 tests unitaires. Ils vérifient notamment :

- l'accès à la liste des produits ;
- la création d'un produit ;
- la conversion du prix en centimes ;
- le retour `404` pour un produit inexistant ;
- le filtrage par catégorie ;
- le rejet d'un prix négatif ;
- le fonctionnement de l'endpoint custom `low-stock` ;
- la mise à jour d'un prix avec recalcul de `price_cents`.

Les tests ont été exécutés avec succès :

```
Found 7 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 7 tests in 0.028s
```

```
OK
Destroying test database for alias 'default'...
```

9 Installation et exécution

9.1 Installation des dépendances

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

9.2 Application des migrations

```
python3 manage.py migrate
```

9.3 Lancement du serveur

```
python3 manage.py runserver
```

9.4 Lancement des tests

```
python3 manage.py test products
```

10 Qualité du projet

Le projet contient les éléments attendus pour un rendu propre :

- un fichier `README.md` clair ;
- un fichier `requirements.txt` listant les dépendances ;
- un fichier `.gitignore` excluant notamment `venv/`, `db.sqlite3`, `__pycache__/` et les fichiers `*.pyc` ;
- des tests unitaires fonctionnels ;
- une documentation Swagger accessible.

11 Conclusion

Le micro-service Catalog Service répond aux critères demandés : endpoints CRUD fonctionnels, respect des conventions REST, documentation OpenAPI, tests unitaires, qualité du projet et fonctionnalités bonus. Les bonus intégrés sont l'endpoint custom avec `@action`, la validation métier customisée et le stockage du prix en centimes.